ASTANA IT UNIVERSITY • DEPARTMENT OF SOFTWARE ENGINEERING
ShP-2216 – Software Design Patterns (OP 6B06102)

Assignment #4: Facade Structural Design Pattern

Student: Nursultan Maratov	**Group:** SE-2523	**Submission Date:** 18.10.2026
Selected Topic: Option B — Online Order Processing Facade	**GitHub Repository:** https://github.com/nursultanmaratov/Assignment4-FacadePattern	

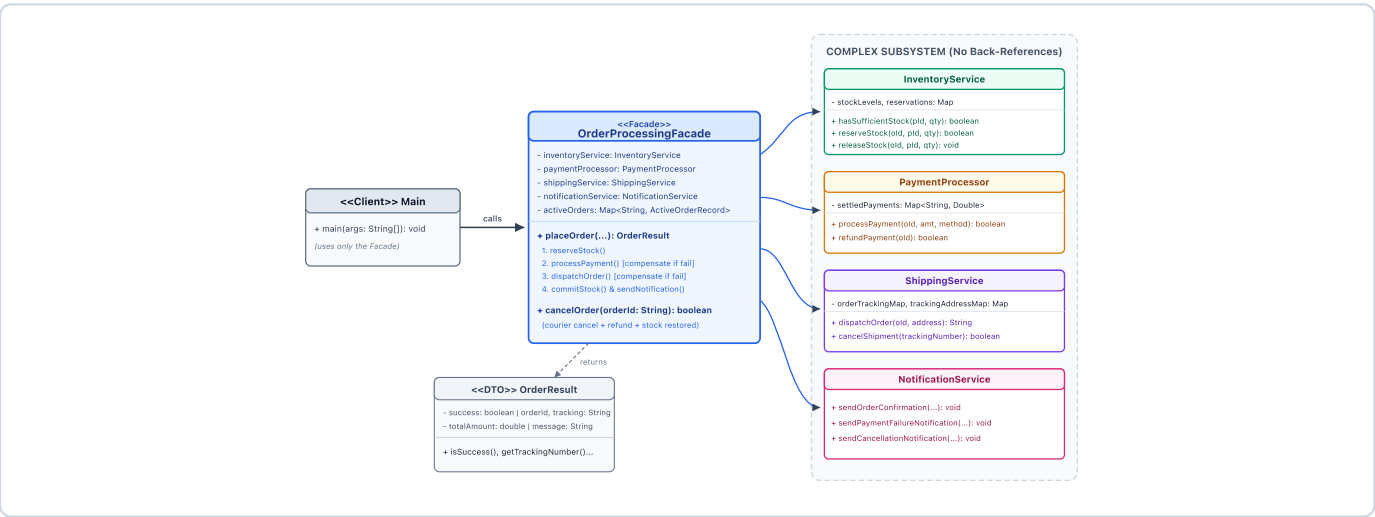
1. Introduction — Chosen Subsystem & Motivation for Facade

In contemporary enterprise systems, placing an e-commerce order is rarely an atomic, single-step operation. Instead, it involves orchestrating multiple heterogeneous, independent subsystem services: verifying stock and locking goods in `InventoryService`, charging credit cards via an external payment gateway in `PaymentProcessor`, booking carrier labels and generating tracking IDs in `ShippingService`, and sending status updates via `NotificationService`.

Why a Facade is required: Without a Facade, the client (e.g. mobile app, web checkout controller, POS terminal) must directly instantiate and manually orchestrate all four services in exact sequence. Furthermore, the client would be forced to handle intermediate transaction failures and execute compensating rollback actions (e.g. releasing reserved warehouse items when a card is declined). This violates information hiding, tightly couples clients to internal service APIs, and leads to error-prone duplication of orchestration logic.

The `OrderProcessingFacade` wraps these four subsystem classes behind a clean, intuitive API (`placeOrder`, `cancelOrder`), completely shielding clients from orchestration complexity, dependency chains, and manual rollback mechanics while keeping subsystem services fully decoupled and accessible to advanced clients.

2. UML Class Diagram



3. Clean Code Principles Justification (Section 4)

Principle 1: Meaningful & Intention-Revealing Names

Method and class names reveal business intent rather than mechanistic operations. The Facade exposes two simple, highly communicative entry points: `placeOrder(...)` and `cancelOrder(...)`. Subsystem methods clearly express what they do (`reserveStock`, `releaseStock`, `processPayment`, `dispatchOrder`).

```
// BEFORE (Ambiguous procedural naming):
boolean status = facade.execute("ORD-1", "LAPTOP", 2, 1200.0, "c@m.com", "Addr", "VISA");
facade.rollbackOp("ORD-1");

// AFTER (Clean, intention-revealing API):
OrderResult result = facade.placeOrder(orderId, productId, quantity, price, email, address, method);
boolean isCancelled = facade.cancelOrder(orderId);
```

✓ Benefit: Client code reads like human prose; call sites are self-documenting.

Principle 2: Single Responsibility Principle (SRP)

The Facade is strictly an **orchestrator**. It delegates domain mechanics to their respective experts and never duplicates calculation or business algorithms inside itself.

```
// ANNOTATED CODE EXCERPT (OrderProcessingFacade.java):
public OrderResult placeOrder(...) {
    // 1. Inventory responsibility stays in InventoryService
    if (!inventoryService.reserveStock(orderId, productId, quantity)) { ... }

    // 2. Financial billing stays in PaymentProcessor
    if (!paymentProcessor.processPayment(orderId, totalAmount, paymentMethod)) { ... }

    // 3. Courier logistics stays in ShippingService
    String trackingNumber = shippingService.dispatchOrder(orderId, shippingAddress);

    // 4. Communication stays in NotificationService
    notificationService.sendOrderConfirmation(customerEmail, orderId, trackingNumber, totalAmount);
}
```

✓ Benefit: High cohesion and low coupling; changes to payment gateway APIs never touch inventory or facade logic.

Principle 3: No Leaking of Subsystem Details through Public Interface

Subsystem internal maps, reservation locks, carrier-specific tracking address records, and intermediate states remain strictly encapsulated. The client receives a clean, immutable `OrderResult` DTO.

```
// BEFORE (Leaking subsystem internals to client):
Map<String, Object> rawData = facade.placeOrder(...);
boolean success = (Boolean) rawData.get("INVENTORY_LOCK_SUCCESS");
String rawGatewayCode = (String) rawData.get("STRIPE_RESPONSE_CODE");

// AFTER (Strict Encapsulation via OrderResult DTO):
OrderResult result = facade.placeOrder(...);
if (result.isSuccess()) {
    System.out.println("Tracking: " + result.getTrackingNumber());
}
```

✓ Benefit: Subsystem implementations can be completely refactored or replaced without affecting clients.

Principle 4: Small, Focused Subsystem Classes with Zero Back-References

Each subsystem service is concise, focused on one conceptual responsibility, and completely ignorant of the Facade's existence. This preserves the defining property of the pattern: advanced clients can still access subsystem classes directly.

```
// Advanced client (e.g. Warehouse Inventory Manager) directly using subsystem:
InventoryService directInventory = new InventoryService();
directInventory.addStock("SMARTPHONE-X", 15);
int total = directInventory.getStock("SMARTPHONE-X"); // Bypasses Facade cleanly
```

✓ Benefit: Subsystems do not degrade into monolithic helper classes; they remain first-class components.

Principle 5: Resilient Error Handling & Compensating Rollback

In distributed multi-step coordination, failure midway through must not leave system state corrupted. When payment declines, the Facade immediately performs compensating actions: releasing the reserved warehouse stock and notifying the customer.

```
// ANNOTATED CODE EXCERPT (Compensating Rollback in OrderProcessingFacade):
boolean paymentSuccess = paymentProcessor.processPayment(orderId, totalAmount, paymentMethod);
if (!paymentSuccess) {
    // Compensating action: restore stock reserved in Step 1
    inventoryService.releaseStock(orderId, productId, quantity);
    notificationService.sendPaymentFailureNotification(customerEmail, orderId, "Payment declined by provider");
```

```
    return OrderResult.failure(orderId, "Payment declined. Reserved inventory has been released.");
}
```

✓ Benefit: Eliminates phantom inventory locks and financial inconsistencies without client intervention.

4. Conclusion — What the Facade Hides vs. What Remains Accessible

What the Facade Hides from Clients	What Remains Directly Accessible
• Multi-step orchestration order across 4 services. • Stock reservation keys and temporary locking logic. • Compensating transactions (automated rollbacks on failure). • Direct dependencies and import coupling to subsystem classes.	• `InventoryService`: warehouse restock, audits, stock queries. • `PaymentProcessor`: accounting audits, independent manual charges/refunds. • `ShippingService`: courier pickups, route updates, tracking lookups. • `NotificationService`: direct transactional notifications.

5. Verification & Test Evidence (5/5 Tests Passed)

Test	Scenario	Verified Behavior	Status
T1	Happy Path Order Placement	Stock reserved (5 → 3), payment $2599.98 settled, tracking `TRK-KZ-1001` assigned, email confirmed.	**PASS**
T2	Insufficient Inventory	Requested 10 units of LAPTOP-PRO (only 3 available). Fast-fail before payment or shipping.	**PASS**
T3	Payment Decline & Rollback	Stock reserved for WIRELESS-BUDS (2 → 0), payment declined. Rollback triggered: stock restored to 2!	**PASS**
T4	Order Cancellation	Cancelled ORD-501: courier dispatch cancelled, $2599.98 refunded, 2 laptops restored (3 → 5).	**PASS**
T5	Direct Subsystem Access	Bypassed Facade to directly restock SMARTPHONE-X (+15 units) via `InventoryService`.	**PASS**